

Cole Boothe C00558477

4/22/26

This project was coded with python using microsofts visual studio code

Part 1.1:

```
server.py > ...
1  import socket
2
3  def server():
4
5      serv_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
6
7      host = '127.0.0.1'
8      port = 12345
9      serv_socket.bind((host, port))
10
11     serv_socket.listen(5)
12     print(f"Server started on {host}:{port}. Waiting for a client...")
13
14     cli_socket, addr = serv_socket.accept()
15     print(f"Connection established with {addr}")
16
17     cli_socket.send("Connection to ClassChat Server successful.".encode('utf-8'))
18
19     try:
20         while True:
21             data = cli_socket.recv(1024).decode('utf-8')
22             if not data:
23                 break
24             print(f"Received from client: {data}")
25
26             response = f"Server received: {data}"
27             cli_socket.send(response.encode('utf-8'))
28
29     except ConnectionResetError:
30         print("Client disconnected.")
31     finally:
32         cli_socket.close()
33         serv_socket.close()
34
35 if __name__ == "__main__":
36     server()
```

This is the code I wrote for part 1.1 to make the server. Line 5 is making the required socket for the server using IPv4 and TCP. Line 8 binds the local port and host connection to the variable named serv_socket. The TCP protocol gets configured in line 5 and gets binded with port number in line 9. The listen and accept for the client are lines 11 and 14, line 14 is also when the cli_socket variable is made because we accept the connection port and address. Line 15 is the acknowledgment of the connection. After connecting with the client there is an infinite loop that checks for new messages, line 21 is when it receives messages and lines 22-23 is to break the loop if there are no

more messages. Line 17 sends a message to the client saying the connection was successful.

Part 1.2:

```
client.py > start_client
1  import socket
2
3  def start_client():
4      cli_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
5
6      serv_ip = '127.0.0.1'
7      serv_port = 12345
8
9      try:
10         cli_socket.connect((serv_ip, serv_port))
11
12         acknowledgement = cli_socket.recv(1024).decode('utf-8')
13         print(f"Server says: {acknowledgement}")
14
15         while True:
16             message = input("Enter message to send (or 'exit' to quit): ")
17             if message.lower() == 'exit':
18                 break
19
20             cli_socket.send(message.encode('utf-8'))
21
22             response = cli_socket.recv(1024).decode('utf-8')
23             print(f"Response from server: {response}")
24
25         except ConnectionRefusedError:
26             print("Error: Could not connect to the server")
27         finally:
28             cli_socket.close()
29
30 if __name__ == "__main__":
31     start_client()
```

Line 4 also does the socket with the same IPv4 and TCP connection. Line 4 also configures the TCP then line 9 configures the address and port number into the client socket. Line 9 is also the line that connects to the server. Lines 12-13 receives and prints the acknowledgment from the server. The while loop that starts on line 15 is a loop that will send messages to the server and also checks if the user wants to disconnect with exit. In the same while loop on lines 22-23 the client receives and prints the responses from the server.

Part 1.3:

These are the server and client terminals communicating to each other. The client UI was slightly changed between screenshots from 1.2 and 1.3.

Server terminal:

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Users/coleb/AppData/Local/Programs/Python/Python39-32/Python.exe -u -i -x -c "import sys; sys.path.append('C:/Users/coleb/AppData/Local/Programs/Python/Python39-32/Python.exe'); import socket; s = socket.socket(socket.AF_INET, socket.SOCK_STREAM); s.bind(('127.0.0.1', 12345)); s.listen(5); print('Server started on 127.0.0.1:12345. Waiting for a client...'); while True: (c, a) = s.accept(); print('Connection established with (' + a + ', ' + str(c.getpeername()[1]) + ')'); data = c.recv(1024).decode('utf-8'); print('Received from client: ' + data); c.send('rah rah'.encode('utf-8')); c.send('test test'.encode('utf-8'))"
```

Client terminal.

```

PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Users/coleb/AppData/Local/Programs/Microsoft VS Code/bin/code .
Type a message
or
Type exit to disconnect

> rah rah
Response from server: Server received: rah rah
> test test
Response from server: Server received: test test
> exit
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project>

```

First message from client was “rah rah” then second message was “test test” then the client disconnected by typing exit.

Part 2:

Updated client

```
client.py > ...
1 import socket
2 import select
3 import msvcrt
4 import sys
5
6 def start_advanced_client():
7     serv_ip = '127.0.0.1'
8     serv_port = 12345
9
10    cli_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
11
12    try:
13        cli_socket.connect((serv_ip, serv_port))
14        cli_socket.setblocking(False)
15        print("\n--- Options ---\n Type a message\n          or\n Hit esc to disconnect\n")
16    except Exception as e:
17        print(f"Connection failed: {e}")
18        return
19
20    inputs = ""
21
22    try:
23        while True:
24            try:
25                readings, _, _ = select.select([cli_socket], [], [], 0.1)
26            except (OSError, ValueError):
27                print("\n Error connecting")
28                break
29
30            if readings:
31                try:
32                    data = cli_socket.recv(1024).decode('utf-8')
33                    if not data:
34                        print("\n Disconnected from the server")
35                        break
36
37                    print(f"\n[Server]: {data}")
38
39                    sys.stdout.write("> " + inputs)
40                    sys.stdout.flush()
41
42                except BlockingIOError:
43                    pass
44
45                if msvcrt.kbhit():
46                    char = msvcrt.getche().decode('utf-8')
47                    if char == '\n':
48                        if inputs.strip():
49                            cli_socket.setblocking(True)
50                            cli_socket.send(inputs.encode('utf-8'))
51                            cli_socket.setblocking(False)
52
53                            print(f"\nSent: {inputs}")
54
55                            inputs = ""
56                            sys.stdout.write("> ")
57                        elif char == '\x1b':
58                            break
59                        else:
60                            inputs += char
61
62            finally:
63                cli_socket.close()
64
65    if __name__ == "__main__":
```

Since on windows to use select() requires a socket to be valid and active, I used the import msvcrt library for the keyboard input. It now uses the select() to activate the client when it receives data from the server. The 0.1 in line 26 is what keeps the cpu from 100% usage.

Server terminal:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
● PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Users/c
Server started on 127.0.0.1:12345. Waiting for a client...
Connection established with ('127.0.0.1', 63584)
Received from client: rah rah
Received from client: test test
○ PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project>
```

Client terminal:

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Users/c
--- Options ---
Type a message
or
Hit esc to disconnect

rah rah
Sent: rah rah
>
[Server]: Server received: rah rah
> test test
Sent: test test
>
[Server]: Server received: test test
>
○ PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> █
```

Part 3:

Updated server

```
server.py > ...
1  import socket
2  import threading
3
4
5  def directing_client(cli_socket, addr):
6      print(f"[NEW CONNECTION] {addr} connected.")
7
8      cli_socket.send("Connected to ClassChat Server".encode('utf-8'))
9
10     connected = True
11     while connected:
12         try:
13
14             message = cli_socket.recv(1024).decode('utf-8')
15             if not message:
16                 break
17
18             print(f"[{addr}] {message}")
19
20             cli_socket.send("Message received".encode('utf-8'))
21
22         except ConnectionResetError:
23             break
24
25     cli_socket.close()
26     print(f"{addr} disconnected.")
27
28
29
30 def server():
31
32     serv_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
33
34     serv_IP = '127.0.0.1'
35     serv_port = 12345
36     serv_socket.bind((serv_IP, serv_port))
37
38     serv_socket.listen(5)
39     print(f"Server started : {serv_IP}:{serv_port}.")
40
41     while True:
42
43         cli_socket, addr = serv_socket.accept()
44
45         thread = threading.Thread(target=directing_client, args=(cli_socket, addr))
46         thread.start()
47         print(f"[ACTIVE CONNECTIONS] {threading.active_count() - 1}")
48
49 if __name__ == "__main__":
50     server()
```

The updated server is of the thread + socket method. Added a new def that handles the client threads. In the server def it communicates with the def that handles the clients. Here is an example of the terminals working.

Server terminal:

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Use
Server started : 127.0.0.1:12345.
[NEW CONNECTION] ('127.0.0.1', 50864) connected.
[ACTIVE CONNECTIONS] 1
[NEW CONNECTION] ('127.0.0.1', 50866) connected.
[ACTIVE CONNECTIONS] 2
[('127.0.0.1', 50864)] teasing teasing
[('127.0.0.1', 50866)] test test
[('127.0.0.1', 50866)] rahhh
('127.0.0.1', 50866) disconnected.
('127.0.0.1', 50864) disconnected.
█
```

Client 1:

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C
--- Options ---
Type a message
or
Hit esc to disconnect

[Server]: Connected to ClassChat Server
> teasing teasing
Sent: teasing teasing
>
[Server]: Message received
>
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> █
```

Client 2:

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:
--- Options ---
Type a message
or
Hit esc to disconnect

[Server]: Connected to ClassChat Server
> test test
Sent: test test
>
[Server]: Message received
> rahhh
Sent: rahhh
>
[Server]: Message received
>
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> █
```

Part 4:

Updated server

```
server.py > directing_client
1  import socket
2  import threading
3  import sys
4  import threading
5  import json
6
7  client_names = {}
8
9  def directing_client(cli_socket, addr):
10
11     username = None
12     try:
13         # receives the first data which is the username then decodes it back into a string
14         initial_data = cli_socket.recv(1024).decode('utf-8')
15         # turns the string in to a python dictionary
16         registration = json.loads(initial_data)
17         # gets the username from the dictionary
18         username = registration.get("sender")
19         # saves the connection object in the dictionary
20         client_names[username] = cli_socket
21
22         # notifies that the new user has joined the server
23         print(f"[REGISTERED] {username} joined the ClassChat.")
24
25         # a loop that is listening for new messages
26         while True:
27             # the data can be upto 10KB
28             data = cli_socket.recv(10240).decode('utf-8')
29             # if no data that means the client is disconnected and breaks the loop
30             if not data: break
31
32             # turns the incoming data back into a python dictionary
33             payload = json.loads(data)
34             receiver = payload.get("receiver")
35
36             # if the receiver matches a name in the dictionary then the string is sent to that user
37             if receiver in client_names:
38                 client_names[receiver].send(data.encode('utf-8'))
39
40             # if the receiver is neither ALL or a matching name then it sends an error code back to client
41             else:
42                 # making error code string
43                 error = json.dumps({
44                     "sender": "Server",
45                     "text": f"User {receiver} is not in the system."
46                 })
47                 # sending error string
48                 cli_socket.send(error.encode('utf-8'))
49
50             # error exception
51         except Exception as e:
52             print(f"[ERROR] {e}")
53         # if the user logs out or unexpectedly disconnects then they are deleted from the dictionary
54         finally:
55             # checking for user in dictionary
56             if username in client_names:
57                 # deleting said user
58                 del client_names[username]
59                 # closing connection
60                 cli_socket.close()
61             # print notifying the server
62             print(f"[DISCONNECTED] {username} left.")
63
```



```

64
65 def server():
66
67     serv_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
68
69     serv_IP = '127.0.0.1'
70     serv_port = 12345
71     serv_socket.bind((serv_IP, serv_port))
72
73     serv_socket.listen()
74     print(f"ClassChat server is live")
75
76     while True:
77
78         cli_socket, addr = serv_socket.accept()
79
80         thread = threading.Thread(target=directing_client, args=(cli_socket, addr))
81         thread.start()
82
83 if __name__ == "__main__":
84     server()

```

The server was updated to read JSON and the thread at the top is now able to read the inputs.

Updated client

```

client.py > client
1  import socket
2  import threading
3  import json
4
5  def handler_direction (cli_socket):
6      while True:
7          try:
8              # receives the data from the server
9              data = cli_socket.recv(10240).decode('utf-8')
10             #if no data break the loop
11             if not data: break
12             # turn into a dictionary
13             payload = json.loads(data)
14             #gets the sender from the dictionary
15             sender = payload.get("sender")
16             text = payload.get("text")
17             # prints the message on the receivers end
18
19             print(f"\n<{sender}>: {text}")
20
21             # allow the user to type again
22             print("> ", end="", flush=True)
23
24         except:
25             break
26
27
28  def client():
29      client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
30      client.connect(('127.0.0.1', 12345))
31
32      # asks for the users username upon first use
33      my_name = input("Enter your ClassChat username: ")
34      # sends the username back to server in JSON format and this is the initial data it was waiting on
35      client.send(json.dumps({"sender": my_name}).encode('utf-8'))
36
37      # starts the handler_direction and will stop when the program closes
38      threading.Thread(target=handler_direction, args=(client,), daemon=True).start()
39
40      # prompts the user with their options
41      print("\n--- Options ---")
42      print("Private: <Name>: <message>")
43      print("Disconnect: exit")
44
45      # loop for the input
46      while True:
47          user_input = input("> ")
48
49          # checks if the user input exit and disconnects from the server
50          if user_input.lower().strip() == "exit":
51              print("Disconnecting from ClassChat")
52              client.close()
53              break
54
55          # reads what the user input
56          if ":" in user_input:
57              target, msg = user_input.split(":", 1)
58              # puts the information into a dictionary
59              payload = {
60                  "sender": my_name,
61                  "receiver": target.strip(),
62                  "text": msg.strip()
63              }
64              # converts the dictionary into a JSON string encodes it and sends it to the server
65              client.send(json.dumps(payload).encode('utf-8'))

```

updated the client to also be able to read JSON and I changed the client to use threads which also allow the user to continue typing when receiving messages.

Running Example:

Server terminal:

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Users/coleb/AppData/Local/ClassChat server is live
[REGISTERED] alice joined the ClassChat.
[REGISTERED] bob joined the ClassChat.
[ERROR] [WinError 10054] An existing connection was forcibly closed by the remote host
[DISCONNECTED] alice left.
[ERROR] [WinError 10054] An existing connection was forcibly closed by the remote host
[DISCONNECTED] bob left.
```

Client 1 (bob) terminal:

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Users/coleb/AppData/Local/ClassChat
Enter your ClassChat username: bob

--- Options ---
Private: <Name>: <message>
Disconnect: exit
> alice: hello
>
<alice>: nice to meet you, but I got to get going
> alice: ok

<Server>: User alice is not in the system.
> > exit
Disconnecting from ClassChat
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> 
```

Client 2 (alice) terminal:

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Users/coleb/AppData/Local/ClassChat
Enter your ClassChat username: alice

--- Options ---
Group: ALL: <message>
Private: <Name>: <message>
Disconnect: exit
>
<bob>: hello
> bob: nice to meet you, but I got to get going
> exit
Disconnecting from ClassChat
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> 
```

For part 4 I ended using ai to help me understand the JSON inputs and how to work them.

Part 5.1:

```
34 # turns the incoming data back into a python dictionary
35 payload = json.loads(data)
36 receiver = payload.get("receiver")
37 # if the client is sending to all then it loops through every user except sender and sends the message
38 if receiver == "ALL":
39     for user, socket_obj in client_names.items():
40         if user != username:
41             # sends the encoded string to the user socket connection object
42             socket_obj.send(data.encode('utf-8'))
```

```
sender = payload.get("sender")

# if the message was for all label it group, if not label it private
tag = "[GROUP]" if payload.get("receiver") == "ALL" else "[PRIVATE]"
# prints the message on the receivers end
print(f"\n{tag} <{sender}>: {payload.get('text')}")

# allow the user to type again
print("> ", end="", flush=True)
```

For the all group chat feature I added an if statement in the server file that loops through and sends the message to everyone in the dictionary that isn't the sender. In the client file I added two lines that checks and assigns a tag whether the message is for all or private.

Example:

Server terminal:

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Users/coleb/AppData/Local/ClassChat server is live
[REGISTERED] alice joined the ClassChat.
[REGISTERED] bob joined the ClassChat.
[ERROR] [WinError 10054] An existing connection was forcibly closed by the remote host
[DISCONNECTED] alice left.
[ERROR] [WinError 10054] An existing connection was forcibly closed by the remote host
[DISCONNECTED] bob left.
```

Client 1(bob):

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Use
Enter your ClassChat username: bob

--- Options ---
Group:  ALL: <message>
Private: <Name>: <message>
Disconnect: exit
>
[GROUP] <alice>: hello everyone
>
[PRIVATE] <alice>: bro I just used the all feature
> alice: that's awesome
> exit
Disconnecting from ClassChat
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> []
```

Client 2 (alice):

```
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> & C:/Users
Enter your ClassChat username: alice

--- Options ---
Group:  ALL: <message>
Private: <Name>: <message>
Disconnect: exit
> ALL: hello everyone
> bob: bro I just used the all feature
>
[PRIVATE] <bob>: that's awesome
> exit
Disconnecting from ClassChat
PS C:\Users\coleb\OneDrive\Desktop\Code\413 semester project> []
```